

Realtime-Venus: A full-duplex interaction system with asynchronous delegation

Venus Team, Ant Group
Tsinghua University

Natural interaction in digital and physical environments requires continuous perception and timely responses. Spoken dialogue relies on acoustic and linguistic cues, while video interaction also requires grounding the conversation in evolving visual context. We present Realtime-Venus, a proactive full-duplex interaction system with two separately trained 9B models: **Realtime-Venus-Omni** for audio-visual interaction and **Realtime-Venus-Audio** for spoken interaction. Each model serves as a complete conversational frontend, integrating continuous perception, conversational control, and native speech generation through a shared causal timeline for user inputs, model outputs, and delegation events. A dual-loop runtime coordinates live interaction with background reasoning and tool execution. Foreground interaction continues while **Realtime-Venus-Harness** executes tasks asynchronously and returns results for integration into the ongoing dialogue. Both models follow a common post-training recipe combining offline understanding, proactive full-duplex trajectories, and delegation workflows. Among the evaluated online models, Realtime-Venus-Omni achieves the highest scores on six of eight video benchmarks, including StreamingBench (70.2%), OVO-Bench (64.7%), and Daily-Omni (81.3%). Across eight audio understanding and spoken question answering benchmarks, Realtime-Venus-Audio leads the compared models on MMAU (78.0%), MMAU-Pro (63.2%), Llama Questions (83.8%), and Speech CMMLU (67.8%), while matching the best VoiceBench AlpacaEval score of 4.81. On Full-Duplex-Bench v1.5, Realtime-Venus-Audio responds to 75% of user interruptions and achieves continuation rates of 97%, 88%, and 86% under backchannels, other-directed speech, and background speech, respectively, exceeding Gemini 3.1 Live and GPT-4o on all three continuation metrics.

Project: <https://realtime-venus.github.io/> **Model:** 🤖 🧠

Code: <https://github.com/inclusionAI/Realtime-Venus>

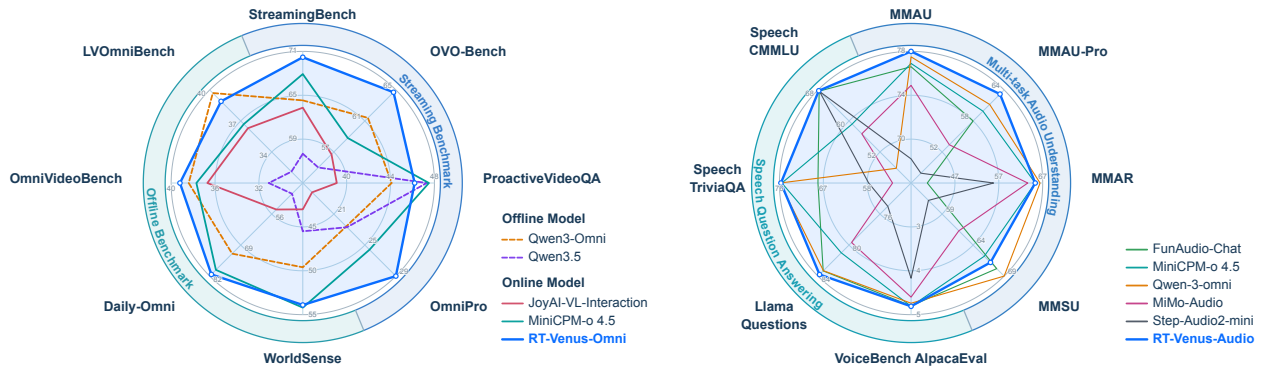


Figure 1 Understanding performance of Realtime-Venus.

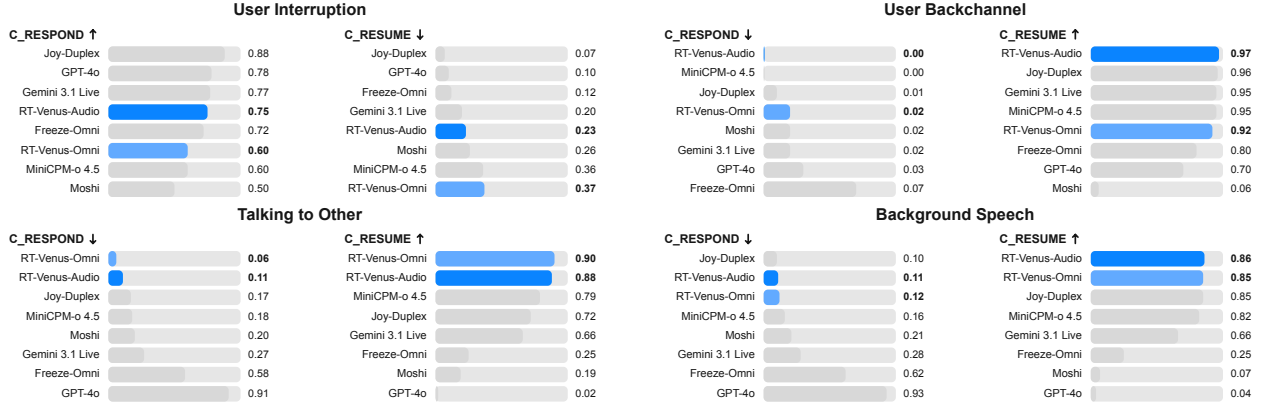


Figure 2 Full-duplex interaction performance of Realtime-Venus.

1 Introduction

Natural interaction requires systems to interpret ongoing observations while deciding when and how to respond, including when to initiate a response without an explicit user request. Recent models increasingly integrate perception, speech generation, and conversational control. Moshi supports concurrent speech modeling (Défossez et al., 2024); Qwen2.5-Omni and Qwen3-Omni combine multimodal perception with native streaming speech generation (Xu et al., 2025a,b); and MiniCPM-o 4.5 extends these capabilities to proactive full-duplex video interaction (Cui et al., 2026). Research on spoken agents explores retrieval, tool calls, and asynchronous external computation during dialogue (Chien et al., 2026; Huang et al., 2026; OpenAI, 2026; Zhang et al., 2026).

Continuous interaction and external computation operate on different timescales within the same session. A background task requires a stable record of the request and its supporting evidence, but its result must be interpreted in a conversation that may have changed during execution. Coordinating task capture with conversational result delivery is essential to maintaining coherent interaction.

We introduce Realtime-Venus, a proactive full-duplex interaction system that combines native conversational modeling with asynchronous delegation. Built on MiniCPM-o 4.5 (Cui et al., 2026), the system provides two separately trained models: **Realtime-Venus-Omni** for audio-visual interaction and **Realtime-Venus-Audio** for spoken interaction. Each serves as a complete conversational frontend with continuous perception, conversational control, and native speech generation. A unified streaming formulation aligns user observations, model outputs, private delegation requests, and background results on a shared causal timeline. Here, the frontend jointly predicts interaction-control tokens, response text, and delegation requests. The shared policy supports maintaining a response during user backchannels, revising its unspoken continuation after a correction, and initiating background work when a request requires external capabilities.

We pair these models with **Realtime-Venus-Harness**, a shared framework for asynchronous capability execution and result delivery. Within a dual-loop runtime, either frontend maintains live interaction while Realtime-Venus-Harness manages delegated work in the background. Each delegation request is bound to its originating session and committed with a snapshot of the evidence available when the request began. The framework routes the task to a registered capability for asynchronous execution and returns eligible results as private context. Freshness checks determine result eligibility, while playback-aware delivery governs when results re-enter the conversation. The frontend then interprets the returned information in the context of the ongoing dialogue and

determines the user-facing response. This design preserves stable context for background execution while allowing the frontend to adapt its response to subsequent changes in user intent.

Training requires trajectories that connect conversational events with interaction decisions and delegated execution. We develop a unified data pipeline combining scenario planning, speech realization, and temporal alignment. Duplex scenarios distinguish backchannels and other-directed speech from interruptions that require stopping, repairing, or redirecting a response. Proactive trajectories supervise when to initiate a response and when to continue listening. Delegation scenarios connect private requests with background execution, returned information, and subsequent responses. The pipeline combines these behaviors within conversations on a common timeline.

The resulting trajectories support a shared post-training recipe that mixes offline understanding, proactive duplex interaction, and delegation workflows. Realtime-Venus-Omni uses both audio-visual and audio-only data, whereas Realtime-Venus-Audio uses the audio-only subset. Supervision covers interaction-control transitions and subsequent generation, linking decisions about when to listen, speak, or delegate to the response that follows.

The evaluations assess understanding, conversational continuity, and delegation. Realtime-Venus-Omni leads the evaluated online models on six of eight video benchmarks, while Realtime-Venus-Audio achieves the highest scores in several audio understanding and spoken question answering comparisons. Full-duplex evaluations show high continuation rates under non-interruptive speech. Complementary tool-use and delegation-decision evaluations distinguish correct routing from successful task completion and identify challenges in executing external work during conversation.

Our contributions are summarized as follows:

- **Proactive full-duplex interaction models.** We develop **Realtime-Venus-Omni** and **Realtime-Venus-Audio**, two separately trained 9B models for proactive audio-visual interaction and full-duplex spoken dialogue, respectively. Both models integrate continuous perception, conversational control, native speech generation, and private delegation under a unified streaming formulation. To our knowledge, Realtime-Venus-Omni is the first full-duplex omni model to support asynchronous backend invocation for reasoning and tool execution while maintaining video interaction. With memory augmentation, it also supports hour-scale video understanding.
- **Asynchronous capability execution with Realtime-Venus-Harness.** We introduce a shared execution framework that binds tasks to evidence available at the request boundary, executes registered capabilities asynchronously, and returns results to the originating session while preserving frontend control over conversational responses.
- **A coupled duplex and delegation data pipeline.** We develop a pipeline combining scenario planning, speech realization, and temporal alignment to construct trajectories coupling conversational events with delegation requests, background results, and response continuations.

2 Related Work

Omni-modal understanding. Omni models integrate text, vision, and audio within shared architectures for understanding and response generation. Gemini (Gemini Team, Google, 2023) learns from interleaved multimodal data for cross-modal understanding and reasoning, while GPT-4o (OpenAI, 2024) supports native speech interaction through end-to-end training across text, vision, and audio. Open models extend these capabilities: Baichuan-Omni-1.5 (Li et al., 2025) combines multimodal

understanding with end-to-end speech generation, and MiniCPM-o 2.6 (OpenBMB, 2025) supports continuous audio–visual inputs and streaming speech through online modality processing and time-division multiplexing. Qwen2.5-Omni (Xu et al., 2025a) introduces time-aligned audio–visual representations and a Thinker–Talker architecture, while Qwen3-Omni (Xu et al., 2025b) extends this design with mixture-of-experts components and multi-codebook speech generation. These advances provide modality coverage and streaming output; full-duplex interaction requires incoming observations to influence an ongoing response.

Audio understanding and speech generation. Audio-language models use complementary approaches to connect acoustic representations with language models. SALMONN (Tang et al., 2024) connects speech and general audio encoders to a large language model (LLM) through a window-level Q-Former. Qwen-Audio (Chu et al., 2023) unifies diverse audio understanding tasks through multitask pretraining with hierarchical task tags, while Qwen2-Audio (Chu et al., 2024) uses natural-language prompts to support spoken instructions and text-guided audio analysis. SpeechGPT (Zhang et al., 2023) extends audio-to-text understanding to speech generation by integrating discrete speech representations into an LLM through modality adaptation and cross-modal instruction tuning. More recent models refine this integration: Kimi-Audio (Ding et al., 2025) combines continuous acoustic features with discrete semantic tokens, and MiMo-Audio (Xiaomi LLM-Core Team, 2025) uses patch-based audio encoding and decoding with next-token pretraining. Fun-Audio-Chat (Tongyi Fun Team et al., 2025) combines dual-resolution speech representations with staged post-training and model merging, while Step-Audio 2 (Wu et al., 2025) incorporates interleaved text–audio generation, reasoning-oriented reinforcement learning, and external retrieval. Our audio frontend aims to retain acoustic and semantic competence while jointly learning conversational control and delegation.

Proactive and full-duplex interaction. Full-duplex models address conversational timing by processing incoming speech during response generation. Moshi (Défossez et al., 2024) models user and assistant audio as parallel streams and couples text with speech through its Inner Monologue formulation. Freeze-Omni (Wang et al., 2025c) uses an auxiliary classifier on LLM hidden states, trained with a joint state-prediction and language-modeling objective, to predict chunk-level dialogue states and determine whether incoming speech requires interrupting the ongoing response. Fun-Audio-Chat (Tongyi Fun Team et al., 2025) extends joint speech–text modeling with parallel input streams and synthesized concurrent dialogues. JoyAI-Talker (Bai et al., 2026) combines a modular Thinker–Talker architecture with Joy-Duplex for interaction-state prediction and turn control. In multimodal settings, MiniCPM-o 4.5 (Cui et al., 2026) introduces Omni-Flow to align incoming audio–visual streams with outputs for simultaneous perception, speech, and proactive engagement. Video-focused methods study when observations warrant a response: LiveStar (Yang et al., 2025) combines response–silence decoding with memory-aware streaming inference, while MMDuet2 (Wang et al., 2025d) learns response timing and content through multi-turn reinforcement learning. Our formulation extends native streaming interaction by placing conversational control, response generation, and private delegation within a shared policy and timeline.

Tool use and asynchronous delegation. Tool-augmented language models connect reasoning with external capabilities. ReAct (Yao et al., 2023) interleaves reasoning and actions with environmental feedback. Spoken and multimodal systems extend this approach to ongoing interaction. DuplexSLA (Zhang et al., 2026) jointly models speech and a structured action channel for planning, interaction control, and tool calls. MoshiRAG (Chien et al., 2026) provides selective asynchronous retrieval for a full-duplex speech model, while DuplexOmni (Huang et al., 2026) pairs continuous multimodal interaction with a pluggable asynchronous thinking layer. JoyAI-VL-Interaction (Yao et al., 2026) connects proactive visual interaction to background delegation using external automatic

speech recognition (ASR) and text-to-speech (TTS) components. GPT-Realtime (OpenAI, 2025) supports asynchronous function calling, and NemotronLabs VoiceChat (NVIDIA, 2026) provides a separate output channel for tool-calling scripts. We study a private delegation interface for separately trained audio and omni frontends. Realtime-Venus-Harness clarifies execution boundaries by fixing evidence at the request boundary, executing registered capabilities, and returning results to the originating session for integration into the current dialogue.

3 System Overview

Realtime-Venus combines a full-duplex conversational frontend with Realtime-Venus-Harness for asynchronous capability execution and reply preparation. Each session uses either Realtime-Venus-Audio, which processes continuous audio, or Realtime-Venus-Omni, which additionally processes visual inputs. Both share the same delegation interface: the frontend handles live interaction and speech output, while the harness executes delegated tasks and prepares replies. Figure 3 summarizes the architecture.

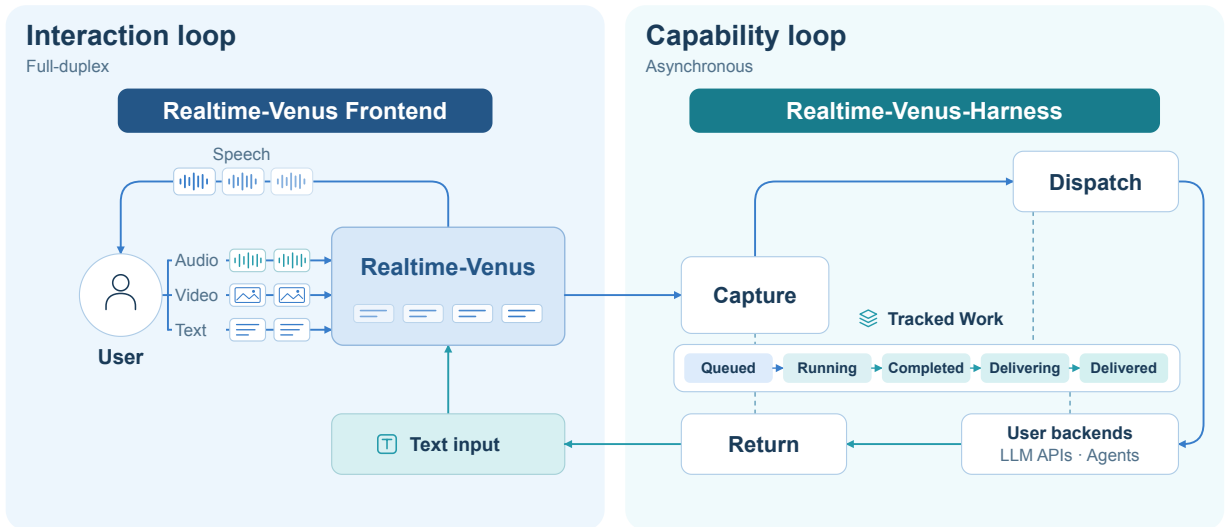


Figure 3 Realtime-Venus runtime: full-duplex interaction with asynchronous delegation.

3.1 Dual-loop architecture

Interaction loop. The frontend continuously processes incoming media, updates the session state, and controls when to listen, speak, or yield. It handles requests that can be answered directly and provides streaming text and speech output through its native Thinker–Talker architecture. Perception remains active during speech and background task execution.

Capability loop. A private natural-language delegation request from the frontend activates this loop. The host hides the request span from user-facing output, captures the context available at the request boundary, and creates an asynchronous work item. The harness selects a backend capability, executes the task, and polishes the result into a reply for spoken delivery. Eligible replies return to the originating session through the private background channel. The frontend chooses when to speak the prepared text during the ongoing interaction; the harness determines its content and wording.

The interaction loop runs continuously and is latency-sensitive, while the capability loop handles tasks on demand over potentially longer durations. Their shared interface exchanges a task package



Figure 4 Example interactions with Realtime-Venus.

bounded by the request’s causal context and a prepared reply bound to the originating session, allowing asynchronous execution alongside live interaction, as illustrated in Figure 4.

3.2 Unified runtime abstraction

Both frontends follow the same runtime structure: they receive media and newly returned replies, update retained session state, and determine interaction behavior and output. Consider a session σ with frontend $m \in \{\text{audio}, \text{omni}\}$, and let k index one-second chunks; session superscripts are omitted. The media inputs are $x_k^{\text{audio}} = u_k$ and $x_k^{\text{omni}} = (u_k, v_k)$, where u_k contains causal audio features and v_k contains aligned visual features, with $v_k = \emptyset$ when no frame is available.

Let s_k denote the frontend state retained before chunk k , including model context, partial output spans, and previously admitted reply text awaiting delivery. The background input b_k contains private reply text newly admitted before assistant generation in that chunk. Previously received replies remain available through the retained state, so their delivery does not require a new background message in every chunk. The runtime feedback η_k records causal events such as playback acknowledgments. The output $O_k = (c_k, Y_k, D_k, S_k)$ comprises interaction-control tokens, foreground text, delegation text, and speech tokens; only Y_k and S_k are user-facing. The causal runtime transition is

$$(s_{k+1}, O_k) = F_m(s_k, x_k^m, b_k; \eta_k). \quad (1)$$

Here, F_m summarizes streaming decoding and runtime scheduling while preserving the causal input–output order within each chunk, as specified in Section 4.3. Delegation text D_k may extend across multiple chunks. A completed request commits a work item whose evidence boundary was fixed when the request began, and its prepared reply can be admitted at a later input boundary. For delegated replies, the frontend uses the current interaction state to schedule speech output of the text prepared by the harness. Section 5 details task capture, capability execution, and reply delivery.

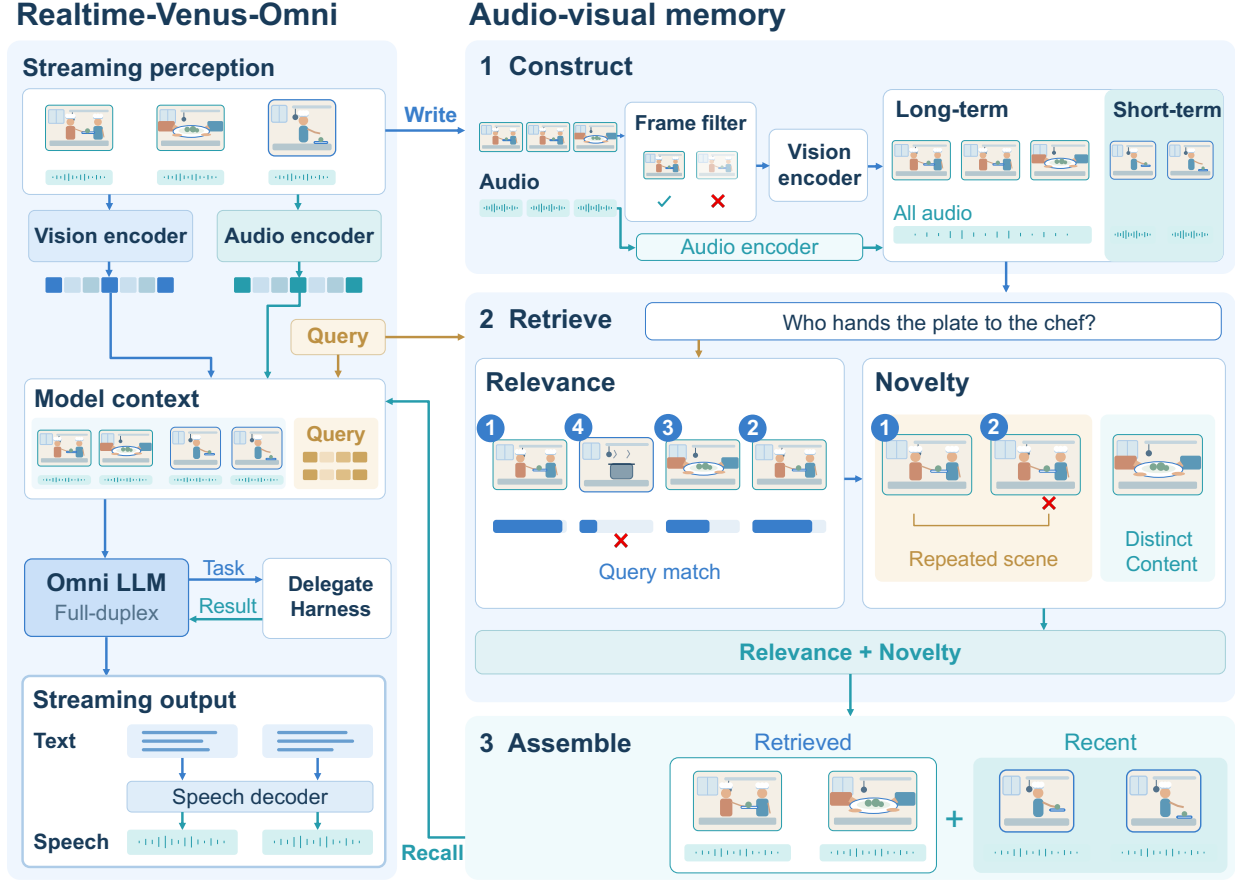


Figure 5 Architecture of Realtime-Venus-Omni for audio-visual perception, full-duplex interaction, and asynchronous delegation.

4 Model Design

Realtime-Venus-Omni and Realtime-Venus-Audio are the omni-modal and audio-only streaming frontends of Realtime-Venus, respectively. Each frontend continuously processes incoming signals, decides whether and when to respond, and generates speech within a single autoregressive interaction loop. We develop both frontends by adapting MiniCPM-o 4.5 (Cui et al., 2026), an open-source 9B model. Both incorporate an in-stream delegation protocol that connects the latency-sensitive foreground loop to the Delegate Harness described in Section 5, enabling complex requests to execute asynchronously while perception and speech generation continue. Realtime-Venus-Omni additionally integrates a training-free long-video memory module that retains relevant visual context across hour-long sessions.

4.1 Model Architecture

The model family inherits the Omni-Flow architecture of MiniCPM-o 4.5, as shown in Figure 5. Realtime-Venus-Omni encodes aligned visual and audio streams with SigLIP2 (Tschannen et al., 2025) and Whisper-Medium (Radford et al., 2023), whereas Realtime-Venus-Audio removes the ViT-based visual branch and processes only streaming audio. The projected features are consumed by a Qwen3-8B language backbone, whose generated text and hidden states condition discrete S3 speech-token prediction. A streaming flow-matching decoder converts these tokens into waveform

chunks using reference audio from the system prompt (Du et al., 2024b).

Streaming is organized into one-second units. Each Realtime-Venus-Omni unit interleaves visual tokens from the current frame with temporally aligned audio features, while each Realtime-Venus-Audio unit contains only audio features. At each unit, the language model predicts `<|listen|>` or `<|speak|>`. For speaking units, response text is generated and used to condition aligned S3 speech-token generation. The token `<|chunk_eos|>` closes a speech chunk, while `<|turn_eos|>` marks the end of an assistant turn. This chunk-wise schedule keeps perception concurrent with speaking and the unspoken continuation revisable, supporting two interaction capabilities: Realtime-Venus-Omni can respond *omni-proactively*—watching and listening continuously and initiating speech when a new event warrants it—while both models conduct *full-duplex conversation*, distinguishing backchannels from interruptions to stop, repair, or redirect the unspoken continuation. These capabilities are supervised by the proactive and full-duplex data in Section 4.5.

4.2 Training-Free Long-Video Memory

Streaming audio-visual input grows continuously, while the base model can maintain only a bounded rolling context window. As a session extends to tens of minutes or even hours, earlier content gradually falls outside the window, making historical information inaccessible to subsequent queries. We therefore attach an external long-term memory module to the streaming pipeline. The module requires no additional training or parameter updates and remains decoupled from the fine-tuned dialogue policy.

Memory construction. Storing the visual representation of every sampled frame would cause the memory size and subsequent retrieval cost to grow continuously over time. Inspired by the predictive visual coding criterion of AdaCodec (Hou et al., 2026), we introduce a visual memory gating mechanism based on motion-compensated prediction cost. Each sampled frame is compared with the preceding sampled frame through lightweight block-level motion matching, and the resulting prediction residual and motion cost are used to estimate visual changes between them. To reduce the risk of discarding short-lived visual events, we further introduce local-change protection and a maximum consecutive-dropping constraint. The gating mechanism only determines whether a sampled frame is archived into long-term memory and does not affect its normal processing within the short-term context. Retained frames are stored together with their visual representations and timestamps, while audio is preserved independently and aligned with the video timeline.

Memory retrieval. Inspired by the MaxSim operator (Khattab and Zaharia, 2020; Wu et al., 2026), the module performs fine-grained matching between query tokens and the visual tokens of each stored historical frame. For each query token, we take its maximum cosine similarity over the visual tokens within a frame as its token-level matching score. To emphasize query tokens whose matching scores vary more across historical frames, we compute the median absolute deviation (MAD) of these scores and normalize the resulting values into token weights. The weighted token-level scores are then aggregated to obtain the semantic relevance score of each frame. Based on the semantic relevance scores, we first identify a set of candidate frames and rank them by relevance. For each candidate, we compare it with higher-ranked candidates to estimate its visual novelty. Inspired by the relevance-diversity principle of Maximal Marginal Relevance (MMR), we combine semantic relevance and visual novelty into a final retrieval score. These scores are computed once based on the initial relevance ranking and remain fixed during selection. The highest-scoring candidates are selected as historical evidence, aiming to preserve query-relevant information while reducing redundant visual content.

Context reassembly. For each retrieved frame, the module retrieves its temporally adjacent audio segments and merges overlapping temporal intervals. The selected historical frames and their associated audio are then arranged chronologically together with the recent short-term audio-visual window and the current query to form the context for answer generation. In this way, the model can recover relevant historical information beyond its rolling context window without continuously retaining the complete input history.

4.3 Unified stream serialization

Realtime-Venus-Omni and Realtime-Venus-Audio represent streaming interaction as a sequence of one-second chunks, with temporal boundary $t_k = k$ s and chunk k covering $[t_{k-1}, t_k)$. Each chunk aligns three logical streams—the user stream, assistant stream, and background stream—on a shared conversational clock, as illustrated in Figure 6.

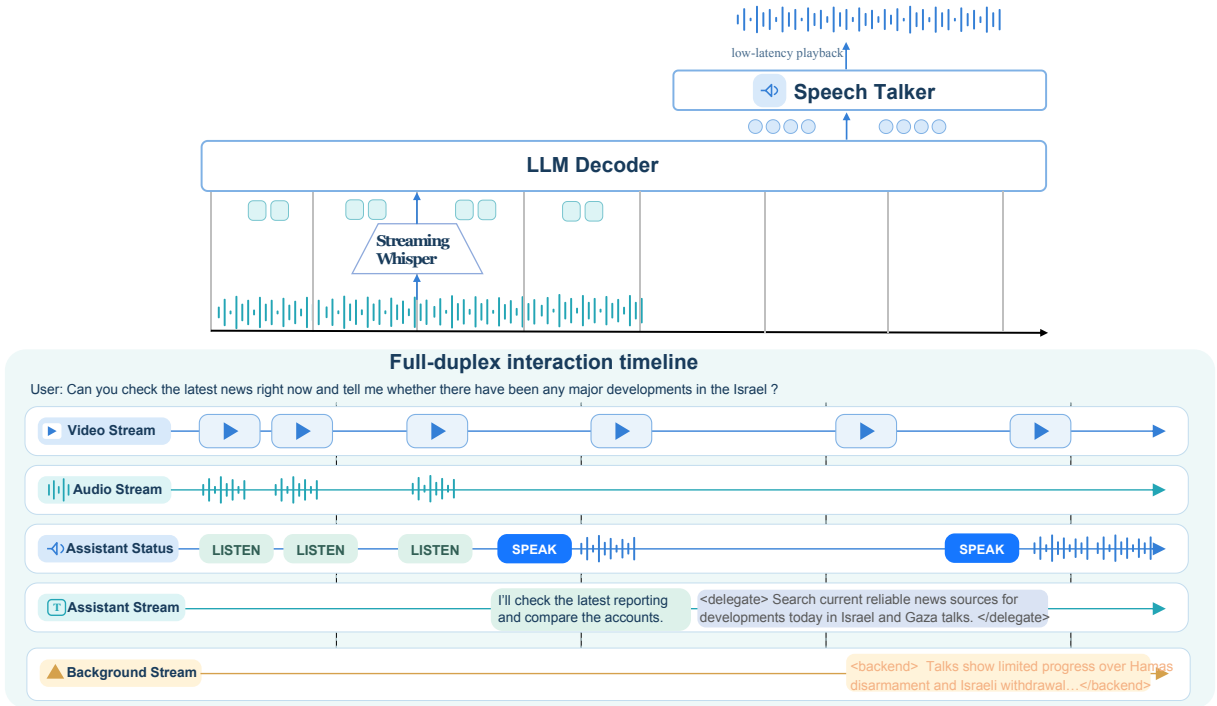


Figure 6 Unified stream serialization for concurrent perception, speech generation, and delegation.

User stream. The user stream is a causal sequence of time-aligned perceptual features. In Realtime-Venus-Omni, projected features from ViT (Dosovitskiy et al., 2020) are interleaved with projected features from Whisper (Radford et al., 2023) at approximately 10 audio features per second; Realtime-Venus-Audio omits the visual features. The stream remains active during assistant output, preserving visual events, pauses, feedback, and overlapping speech on the shared timeline.

Assistant stream. The assistant stream combines foreground text, aligned S3 speech tokens (Du et al., 2024a,b) at about 25 tokens per second, and optional text-only delegation instructions. LLM hidden states preserve contextual prosody during speech generation, while delegation instructions are routed to the backend rather than spoken.

Background stream. The background stream is an asynchronous text-only sequence that connects the interaction models to a more capable backend agent for complex reasoning and tool-based tasks. A delegation instruction launches backend computation, which incurs a variable response delay.

A pending task contributes no result tokens, allowing foreground interaction to continue. Once eligible for delivery, its result is inserted before the assistant stream at an available chunk boundary as external context rather than a prediction target.

The three streams are interleaved into a single token stream consumed by the LLM backbone. The serialization takes the following form:

```

chunk 5: <unit> V A → <listen> </unit>
chunk 6: <unit> V A → <lspeak> T <lchunk_eos><lturn_eos> </unit>
chunk 7: <unit> V A → <lspeak> <delegate> T </delegate><lchunk_eos><lturn_eos> </unit>
chunk 8: <unit> V A → <listen> </unit>
...
chunk 13: <unit> V A → <listen> </unit>
chunk 14: <unit> V A <backend> T </backend> → <lspeak> T <lchunk_eos><lturn_eos> </unit>

```

Notes. Each line represents one unit, written as `<unit> input → output </unit>`. Outputs are shown within one chunk for clarity; in practice, the text output tokens (**T**) of a single turn may span multiple consecutive chunks.

Here, T denotes text tokens, A an audio representation, and V a projected visual representation. The V and A tokens are interleaved according to their arrival times in Realtime-Venus-Omni, whereas Realtime-Venus-Audio serializes only A .

External tool requests appear as delegate spans, and their results appear as background spans on the shared timeline. We add four control tokens—`<delegate>`, `</delegate>`, `<backend>`, and `</backend>`—to the tokenizer, input embeddings, and LM head. When tools are required, the model emits a self-contained natural-language task inside a delegate span, which is hidden from display and speech. The backend agent decomposes the task into one or more function calls and returns eligible results through a background span at an available input boundary. An additional request arriving during assistant speech can therefore be dispatched without closing the foreground turn, while compound requests can trigger multiple calls in semantic order. In both cases, tool execution proceeds asynchronously alongside foreground speech and perception.

Three state tokens represent full-duplex turn control. `<listen>` indicates continued perception without speech, `<lspeak>` marks active foreground generation, and `<lturn_eos>` closes a completed assistant turn. Together, they provide a compact chunk-level interface for interpreting backchannels, distinguishing interruptions, and coordinating transitions between listening and speaking.

To synchronize generated speech with the conversational clock, we adapt text emission to accumulated playback progress. Let τ_{k-1} denote the end time of previously scheduled speech and $\mathcal{D}(Y_{k,1:n})$ the estimated duration of the first n candidate text tokens. At chunk k , we select

$$n_k = \arg \min_n |\tau_{k-1} + \mathcal{D}(Y_{k,1:n}) - t_k|, \quad (2)$$

where t_k is the current chunk boundary and τ_{k-1} carries the accumulated playback progress. The scheduler emits fewer text tokens when speech lags and more when playback capacity is available, aligning the response with the latest user audio and background results. During training, text tokens and their corresponding S3 tokens are assigned to chunks by start time, teaching history-dependent interleaving without a fixed text-speech ratio.

4.4 Full-duplex interaction

We formulate spoken interaction as chunk-level autoregressive modeling over the three synchronized streams. Let $U = (u_1, \dots, u_K)$ denote the causal user-audio features, $B = (b_1, \dots, b_K)$ the sparse

background results available at each chunk, and $O_k = (c_k, Y_k, D_k, S_k)$ the assistant output at chunk k . Here, c_k is an interaction-control token, Y_k is foreground response text, D_k is an optional delegate request, and S_k is the aligned S3 speech-token segment. Fields that are inactive in a chunk are represented as empty sequences.

The LLM with parameters θ predicts interaction control, foreground text, and delegate text, while the speech decoder with parameters ϕ generates speech tokens from the LLM hidden states H . The joint policy factorizes as

$$p_{\theta, \phi}(O_{1:K} \mid U, B) = \prod_{k=1}^K p_{\theta}(c_k, Y_k, D_k \mid U_{\leq k}, B_{\leq k}, O_{< k}) p_{\phi}(S_k \mid H_{\leq k}, Y_{\leq k}, S_{< k}). \quad (3)$$

This factorization keeps acoustic generation outside the main LLM while allowing speech, turn control, and delegation to share the same semantic representation. Background results are external observations rather than model outputs and become available only after the corresponding backend computation finishes.

The control variable c_k determines whether the assistant listens, speaks, or closes its current turn. Because c_k is predicted from the same semantic representation used for response generation, overlapping audio is interpreted according to its conversational role rather than treated as a uniform stop signal. We distinguish the following cases:

- **Pauses and background noise.** Silence, hesitation, or acoustic activity not directed at the assistant produces `<listen>`, keeping perception continuously active without prematurely starting or terminating a response.
- **Backchannels.** A short acknowledgment such as “yes” or “right” does not claim the conversational floor. The model preserves `<lspeak>` and continues the current response plan.
- **Interruptions.** When overlapping user speech takes the conversational floor with a correction, redirection, or new request, the model emits `<turn_eos>` to terminate the stale response. It then incorporates the new intent into its conversational state before generating the next `<lspeak>` segment or delegate request, revising the unspoken continuation based on the new input while leaving already played audio unchanged.

We model interruptions as event-conditioned semantic control and jointly supervise the state transition and subsequent generation trajectory. The *stop/reject* action closes the turn and discards its unspoken remainder; *repair/update* incorporates new evidence and regenerates the stale continuation; and *redirect/follow-up* suspends the current plan and transfers control to a new request. A shared semantic state across turn control, text, speech, and delegate generation enables acoustically similar overlaps to produce distinct continuations, supporting native full-duplex control beyond binary reactions triggered by voice activity detection (VAD).

4.5 Training data

Realtime-Venus-Omni and Realtime-Venus-Audio are trained on a common post-training corpus of over 2.8 million samples covering nine data categories, summarized in Table 1. The corpus is organized into video and audio data. Realtime-Venus-Omni uses both modalities, while Realtime-Venus-Audio is trained only on audio data. The data cover three main capabilities: *offline understanding*, including general audio–visual understanding, general audio understanding, and spoken question answering; *proactive duplex interaction*, including visual-, multimodal-, speech-,

and audio-driven interaction with response timing and interruption handling; and *delegation*, which trains the delegate–backend–restate workflow described in Section 5.

Table 1 Training data composition.

Modality	Category	Size	Role
Video	General AV understanding	936k	offline AV comprehension
	Visual-proactive duplex	454k	visually triggered proactivity
	Omni-proactive duplex	201k	multimodal proactive interaction
	Speech-in duplex	315k	proactive response to in-stream spoken queries
	Delegate	95k	delegate–backend–restate
Audio	General audio understanding	470k	offline audio comprehension
	Spoken question answering	205k	offline spoken question answering
	Speech-only duplex	100k	full-duplex interaction and interruption handling
	Delegate	90k	audio delegation

By modality, the video group constitutes approximately 70% of the corpus and is consumed exclusively by Realtime-Venus-Omni, whereas the audio group accounts for approximately 30% and is shared by Realtime-Venus-Omni and Realtime-Venus-Audio. Offline understanding represents approximately 56% of the corpus, proactive duplex interaction accounts for approximately 37%, and delegation workflows comprise the remaining 6%. Both duplex and delegation sources contain sessions with interruptions, so overlapping speech is supervised as part of ordinary interaction rather than through a separate module. Section 6 describes how the full-duplex and delegate datasets are constructed. Training uses this mixture under the unified recipe described in Section 4.6.

4.6 Training recipe

Realtime-Venus-Omni and Realtime-Venus-Audio use a unified post-training recipe for multimodal streaming. The recipe supports video, audio-only, and text-only inputs and combines full-duplex and offline turn-based formats in a single post-training pass. It mixes proactive duplex data (including speech-in queries), delegation data, and general understanding data, as detailed in Section 4.5. The two variants share the training recipe but differ in input modalities and training-data coverage. The user stream of Realtime-Venus-Omni interleaves projected visual and audio tokens, and the model uses both video and audio data. In contrast, Realtime-Venus-Audio retains only the audio segments and is trained on the audio group.

Objective. Supervision is sparse: loss is computed only on response spans, excluding system, user, and media-placeholder tokens. In offline multi-turn data, assistant history is supervised together with the final turn. For full-duplex data, supervision covers the per-second $\langle \text{listen} \rangle / \langle \text{speech} \rangle$ decision tokens together with the spoken text, and a long reply is supervised continuously across multiple units. Loss is normalized per sample: each sample’s weighted loss is divided by its total supervision weight before averaging over the batch, preventing its contribution from scaling with the number of supervised tokens. For long answers, the LM-head cross-entropy is evaluated in chunks with recomputation, bounding activation memory without changing the objective. Samples that fail to decode or exceed the length budget are discarded during preprocessing and excluded from training. Only the Thinker is updated. The acoustic decoder, which synthesizes audible speech from Thinker hidden states, remains fixed and is excluded from the training objective.

5 Harness Design

Realtime-Venus-Harness connects full-duplex interaction to models and tools for asynchronous reasoning and execution. It provides Realtime-Venus-Omni and Realtime-Venus-Audio with a common execution interface, manages each task’s context, and tracks its progress through delivery to the conversation. Background execution uses evidence fixed at the request boundary, while the frontend interprets the returned result using the conversational state at reintegration.

As shown in Figure 7, Realtime-Venus-Harness comprises three stages: *capture*, *dispatch*, and *return*, with work lifecycle tracking across all three. Capture establishes the task and its evidence scope; dispatch selects and invokes a registered capability; return prepares the result for the originating frontend. A work item preserves task identity and records progress throughout execution and delivery, while foreground perception and interaction continue independently.

Realtime-Venus-Harness

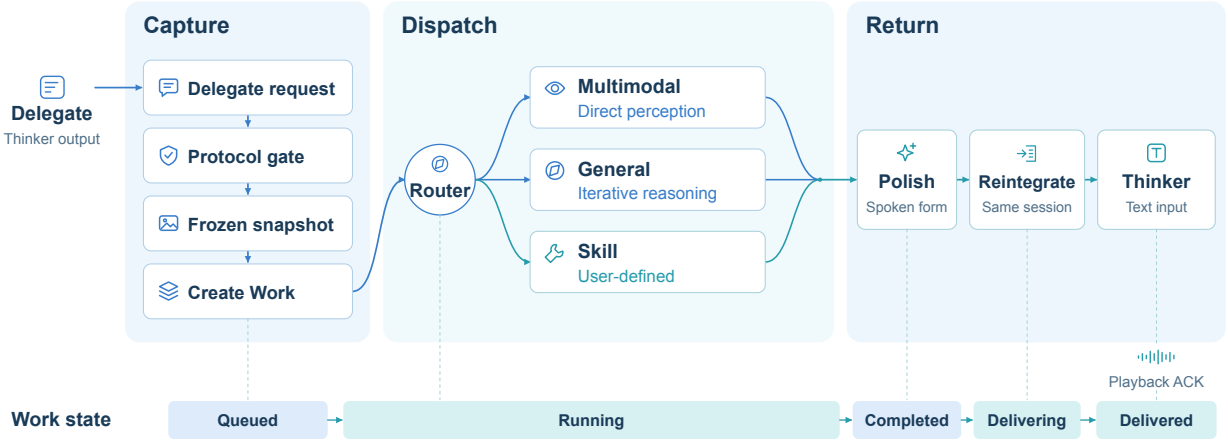


Figure 7 Architecture of Realtime-Venus-Harness for asynchronous task execution and result delivery.

5.1 Causal task capture

The frontend specifies an objective through the private span $\langle \text{delegate} \rangle q_i \langle / \text{delegate} \rangle$. The host extracts this natural-language request from the raw Thinker token stream and suppresses it from visible text and speech. Protocol validation and task-identity checks ensure that an accepted request creates at most one tool work item. The frontend can therefore request external help without encoding backend-specific tool calls or capability schemas.

Let i index accepted request identities in session σ , and let $\kappa_i = (T_i, n_i)$ contain the opening token’s timestamp and the latest input sequence observed at that boundary. Capture uses the bounded session buffer $\mathcal{B}^\sigma$ to construct

$$\begin{aligned} X_i &= \text{Snap}(\mathcal{B}^\sigma; \kappa_i, \Delta), \\ w_i &= P(i, \sigma, q_i, X_i), \end{aligned} \tag{4}$$

where Δ is the look-back horizon and P is a stateful registration operation keyed by request identity. Registration occurs only after the closing token completes q_i . Snapshot timestamps and Δ use a common time base; timestamp and sequence fences restrict evidence to observations available at T_i , with media clipped to the permitted window. Prior assistant speech requires playback confirmation by T_i . Later observations or acknowledgments cannot alter X_i . The work item retains this evidence boundary independently of its commitment time.

5.2 Extensible capability execution

After capture, dispatch assigns each work item to a registered backend capability. A shared execution interface allows the frontend to express a natural-language objective while the harness manages capability selection and execution.

The registry provides two core capabilities and supports extensions through registered skills. The *multimodal* capability uses frozen audio or visual evidence for perception and question answering. The *general* capability returns structured plans for open-ended, multi-step objectives. A registered *skill* invokes a domain executor for task-specific procedures and tools. Each capability has a contract specifying its permitted inputs and argument schema.

Let $\mathcal{C} = \{\gamma_{\text{mm}}, \gamma_{\text{gen}}\} \cup \mathcal{C}_{\text{skill}}$ denote the registry and $\mathcal{A}_\gamma$ the valid argument space of capability γ . Given an objective q_i , the router R makes one model-based decision using the registered contracts and z_i , which contains a compact evidence summary and permitted request metadata. It selects a capability and supplies its arguments without inspecting raw media. A validated decision satisfies

$$(\gamma_i, \alpha_i) = R(q_i, z_i, \mathcal{C}), \quad \gamma_i \in \mathcal{C}, \quad \alpha_i \in \mathcal{A}_{\gamma_i}. \quad (5)$$

Schema validation rejects invalid selections or arguments before execution.

On successful execution, the selected executor produces a normalized result $r_i = E_{\gamma_i}(w_i, \alpha_i)$. This common result format allows all execution paths to feed the same return stage. New skills extend the registry by supplying a capability contract and an executor that follows this interface, preserving the frontend’s delegation protocol and the shared result format.

5.3 Conversational result reintegration

The return stage of Realtime-Venus-Harness prepares the text to be spoken in response to a delegated task. The adapter A applies a shared text-only polishing pass to turn the execution result into a reply, removes reserved control tokens, and wraps the prepared text in a single private `<backend>` message:

$$\tilde{b}_i = A(w_i, r_i), \quad (6)$$

where w_i is the originating work item and r_i is its normalized execution result. The harness determines the content and wording of the reply before handing it to the frontend.

The prepared message returns to the originating session through the private background channel. Messages admitted at chunk k form the background input b_k in Eq. 1. Admission makes the reply available to the frontend as private conversational context, while spoken delivery may occur later. The frontend uses the ongoing interaction to select a suitable time to speak the prepared text and produces speech through the native Talker. This division assigns reply preparation to the harness and conversational timing and speech output to the frontend.

5.4 Work lifecycle and delivery tracking

Successful execution and delivery follow the path

$$\text{QUEUED} \rightarrow \text{RUNNING} \rightarrow \text{COMPLETED} \rightarrow \text{DELIVERING} \rightarrow \text{DELIVERED}.$$

A committed work item initially enters QUEUED; routing, capability execution, and reply polishing occur during RUNNING. The work item enters COMPLETED when the harness has constructed its terminal result, including the prepared reply text. An unrecoverable execution error produces

the terminal state FAILED. The request carries a freshness deadline; a result marked stale at terminal-result construction is excluded from reintegration.

COMPLETED records that the reply has been prepared, while DELIVERING begins when the prepared message is reserved for admission to the originating session. The frontend controls when the reply is spoken. For a speech-bearing reply, DELIVERED requires both completed frontend generation for that reply and playback acknowledgment of every associated Talker chunk. These states distinguish a reply that is ready for delivery from one whose speech has been played.

This lifecycle tracks each work item through execution and delivery, providing a basis for handling exceptions at different stages. When an exception occurs, the recorded state helps the runtime identify the affected stage and determine how to handle the task.

6 Full-duplex and Delegation Data Pipeline

Full-duplex interaction involves concurrent user, assistant, and environmental streams. Acoustically similar events may require different responses depending on their conversational function: a brief backchannel should normally preserve the current response, whereas an interruption may require the assistant to stop and revise it. We therefore represent each session on a shared timeline that records speaker activity, overlap, intended addressee, conversational function, and the state of the active assistant response. The same representation connects interaction events to delegation operations, enabling conversational control and background computation to be supervised within a unified trajectory. This continuous-stream formulation follows prior work on real-time spoken dialogue (Défossez et al., 2024).

6.1 Unified data construction

As illustrated in Figure 8, the pipeline contains three stages: scenario and interaction planning, paralinguistic and acoustic realization, and temporal alignment with model-specific materialization.

Scenario and interaction planning. Each session begins with a structured scenario seed specifying the setting, participants, user goal, language profile, acoustic environment, and target interaction conditions. The seed is expanded into an event-level plan that determines the semantic progression, speaker and addressee, floor ownership, and whether each incoming event preserves or changes the active user intent. Compositional sampling covers daily communication, information seeking, collaborative tasks, decision making, service interactions, and multi-party conversations. The plans include ordinary turn-taking as well as pauses, backchannels, interruptions, corrections, other-directed speech, and background speech.

Paralinguistic and acoustic realization. Planned turns are converted into speech and arranged on the session timeline according to their intended boundaries and overlaps. Hesitation, filled pauses, repetition, self-correction, incomplete utterances, laughter, and breathing are added to improve conversational realism and expose cues relevant to turn-taking. Foreground dialogue is mixed with scene-matched secondary speakers and environmental audio, with event onset and duration varied to produce different overlap conditions. The corpus covers Chinese, English, and code-switching. For audio–visual instances, the composed audio is synchronized with the associated video while retaining the interaction structure and delegation annotations specified by the plan.

Temporal alignment and materialization. Audio, video, text, and system actions are mapped to a common monotonic session clock. Each event records its source, temporal span, conversational

Full-Duplex and Delegate Data Pipeline

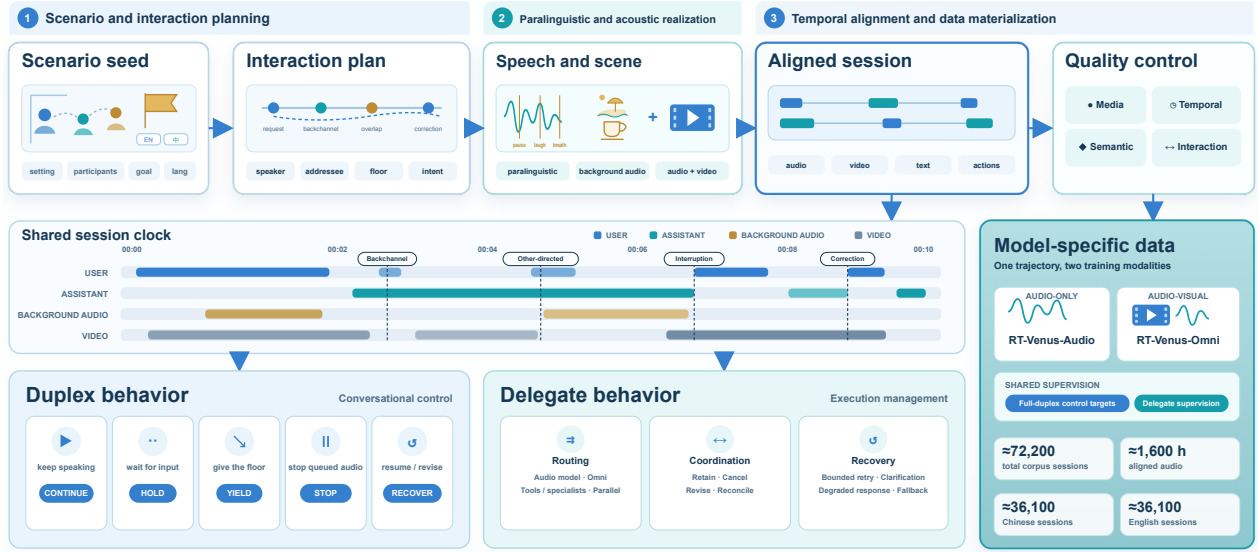


Figure 8 Full-duplex data pipeline.

function, intended addressee, and associated system behavior. The aligned session preserves overlap intervals, response boundaries, and the portion of assistant output affected by an interaction event. It is then materialized as audio-only data for Realtime-Venus-Audio or video data for Realtime-Venus-Omni. Both modalities share the underlying trajectory and supervision: a compact model-visible vocabulary encodes online interaction-control actions, while non-audible delegation requests, background results, and execution metadata remain in the structured event stream.

6.2 Coupling duplex and delegation behaviors

Each trajectory links an observed event to two complementary targets. The **duplex target** specifies the immediate conversational action: continue speaking, await input, yield the floor, stop queued output, or revise the response. Selection reflects conversational meaning, not acoustic overlap alone. Backchannels, other-directed speech, and unrelated background speech generally preserve the active response, whereas floor-taking interruptions stop queued audio. Hesitation may pause responses; intent-changing corrections trigger recovery.

The **delegation target** specifies computational handling. Simple or latency-sensitive conversational acts remain with the frontend. Requests requiring additional perceptual evidence, broader context, or substantial replanning may invoke a registered capability; those involving external information or executable actions are routed to tools or specialized components. Longer operations can run alongside a cancellable local acknowledgment. Supervision specifies whether active work is retained, canceled, revised, reconciled with another result, retried, or replaced by a fallback.

The event’s effect on user intent links both targets. A backchannel preserves speech and computation; other-directed and background speech create no new route. A stable-intent interruption may stop playback while retaining the semantic plan; an intent-changing correction cancels dependent work and triggers replanning. Tool-dependent requests pair immediate feedback with asynchronous execution. Each trajectory connects observable context, interaction control, computation management, and execution outcomes on a shared timeline.

7 Evaluation Protocol

We evaluate Realtime-Venus through complementary assessments of its two frontend models. We assess Realtime-Venus-Omni on streaming and offline video understanding. Both Realtime-Venus-Omni and Realtime-Venus-Audio are evaluated on audio understanding, spoken question answering, and overlap handling on Full-Duplex-Bench v1.5. We also evaluate both frontends within Realtime-Venus on the tool-use component of Full-Duplex-Bench v3 (FDB-v3) and compare their routing decisions on our internally developed delegate benchmark.

7.1 Evaluation datasets and metrics

Streaming and offline video understanding. StreamingBench (Lin et al., 2024) covers 18 tasks in real-time visual, omni-source, and contextual understanding. OVO-Bench (Niu et al., 2025) covers 12 tasks in real-time visual perception, backward tracing, and forward active responding. ProactiveVideoQA (Wang et al., 2025e) reports PAUC across WEB, EGO, TV, and VAD, while OmniPro (Zhao et al., 2026) reports probe-mode accuracy and online-mode F1. For offline video understanding, we report overall accuracy on WorldSense (Hong et al., 2026), Daily-Omni (Zhou et al., 2025), OmniVideoBench (Li et al., 2026), and LVOmniBench (Tao et al., 2026). The memory study reports accuracy by duration bin on LVOmniBench, LongVideoBench (Wu et al., 2024), and CGBench (Chen et al., 2025).

Audio understanding. We use MMAU (Sakshi et al., 2024), MMAU-Pro (Kumar et al., 2025), MMAR (Ma et al., 2025), and MMSU (Wang et al., 2025a) to evaluate acoustic perception and reasoning. MMAU covers speech, music, and sound; MMAU-Pro extends evaluation to challenging long-form, spatial, and multi-audio settings. MMAR emphasizes reasoning from contextual and acoustic evidence, while MMSU assesses spoken-language understanding, including linguistic and paralinguistic cues. We report accuracy under the task configuration used for each benchmark.

Spoken question answering. We evaluate VoiceBench AlpacaEval (Chen et al., 2024), Llama Questions (Nachmani et al., 2024), Speech TriviaQA (Défossez et al., 2024), and Speech CMMLU (Shi et al., 2026). These tasks assess instruction following, factual knowledge, and Chinese academic knowledge through spoken queries. VoiceBench AlpacaEval uses a judge-based response-quality score; the remaining tasks use benchmark-specific measures of answer accuracy.

Full-duplex interaction. Full-Duplex-Bench v1.5 (Lin et al., 2026b) evaluates user interruption, user backchannel, talking to others, and background speech. Its four response categories are C_RESPOND, C_RESUME, UNCERTAIN, and UNKNOWN, denoting responses to overlapping speech, continuation of the preceding response, expressed uncertainty, and irrelevant or absent responses. Responding is the desired behavior for user interruptions; continuing the preceding response is preferred in the other scenarios.

Full-duplex tool use. We evaluate the tool-use component of Full-Duplex-Bench v3 (FDB-v3) (Lin et al., 2026a), which tests voice agents on human-recorded speech containing fillers, pauses, hesitations, false starts, and self-corrections. Its tasks span four application domains and require single-step or chained calls to deterministic mock APIs. Tool selection F1 compares expected and predicted tool calls, penalizing missed and extra calls. Argument accuracy measures the semantic correctness of function arguments. Pass@1 is the proportion of episodes in which all expected tools are called, no extra calls are made, and every call has correct arguments. We report these metrics as percentages.

Full-duplex delegation. We evaluate delegation decisions using an internally constructed delegate

benchmark. It includes omni and audio-only input modes with different levels of difficulty. Its tasks span three categories: external capabilities, routine interaction, and reasoning. Overall accuracy measures agreement with reference delegation decisions across all requests within each mode. Delegation recall measures the proportion of external-capability requests correctly delegated. Non-delegation specificity measures the proportion of routine requests for which the model correctly avoids delegation. Reasoning accuracy measures correct routing on a mixture of requests requiring delegation and direct handling. We report these metrics as percentages for each input mode.

7.2 Evaluation settings

By default, we sample videos at 1 frame per second, retain at most 128 frames, and resize frames to approximately 448×448 pixels while preserving the original aspect ratio. Streaming predictions are conditioned on observations available at the corresponding query or response time, while offline tasks permit access to the sampled evaluation clip before answering. For both Realtime-Venus-Omni and Realtime-Venus-Audio, input waveforms are converted to mono and resampled to 16 kHz. During full-duplex evaluation, incoming audio is processed in one-second chunks, with perception remaining active during speech generation. When speech output is required, waveforms are generated at 24 kHz. We report offline and online baselines separately in the summary table. In all tables, **bold** and underlined values indicate the best and second-best distinct scores, respectively.

7.3 Omni results

Streaming and offline understanding. Table 2 compares Realtime-Venus-Omni with offline and online baselines across eight benchmarks.

Table 2 Streaming and offline video understanding results.

Model	Size	Streaming benchmarks				Offline benchmarks			
		Streaming Bench	OVO-Bench	Proactive VideoQA	OmniPro	WorldSense	Daily-Omni	OmniVideo Bench	LVOmni Bench
Offline models:									
InternVL3.5	8B	60.8	53.8	37.5	12.1	39.2	53.4	35.7	38.1
Qwen3-VL	8B	53.5	49.8	43.4	19.5	44.0	45.6	30.7	30.2
Qwen3.5	9B	57.0	55.0	47.3	22.7	45.5	47.5	31.1	32.1
Qwen3-Omni	30B	64.3	61.4	44.1	22.6	49.6	72.6	38.4	39.7
video-SALMONN 2+	7B	59.9	46.8	40.4	22.1	49.8	66.1	35.8	38.7
AV-Flamingo	7B	58.5	51.5	34.6	18.6	53.2	69.4	39.9	38.0
Gemini-3.5-Flash	–	77.3	71.6	56.5	52.4	67.2	82.3	64.6	58.2
Online models:									
LiveStar	8B	60.2	38.0	25.0	8.3	36.1	44.7	29.3	32.5
MMDuet2	3B	58.1	47.6	34.9	7.6	36.9	50.6	30.4	32.1
JoyAI-VL-Interaction	8B	63.3	56.7	39.1	18.2	43.0	54.1	36.7	36.3
MiniCPM-o 4.5	9B	<u>67.9</u>	<u>60.7</u>	47.5	<u>25.6</u>	54.2	<u>79.4</u>	<u>37.7</u>	<u>36.7</u>
Realtime-Venus-Omni	9B	70.2	64.7	<u>46.2</u>	29.0	<u>54.0</u>	81.3	39.2	38.9

Notes. StreamingBench, OVO-Bench, WorldSense, Daily-Omni, OmniVideoBench, and LVOmniBench report accuracy. ProactiveVideoQA reports PAUC and OmniPro reports probe-mode accuracy. All scores are shown on a 0–100 scale; offline models are shown for reference.

The offline baselines are InternVL3.5 (Wang et al., 2025b), Qwen3-VL (Bai et al., 2025), Qwen3.5 (Qwen Team, 2026), Qwen3-Omni (Xu et al., 2025b), video-SALMONN 2+ (Tang et al., 2025), AV-Flamingo (Ghosh et al., 2026), and Gemini-3.5-Flash (Google DeepMind, 2026). The online baselines are LiveStar (Yang et al., 2025), MMDuet2 (Wang et al., 2025d), JoyAI-VL-Interaction (Yao et al., 2026), and MiniCPM-o 4.5 (Cui et al., 2026).

Results. Realtime-Venus-Omni scores 70.2 on StreamingBench, 64.7 on OVO-Bench, 46.2 on ProactiveVideoQA, and 29.0 on OmniPro. Its offline video understanding scores are 54.0 on WorldSense, 81.3 on Daily-Omni, 39.2 on OmniVideoBench, and 38.9 on LVOmniBench. Compared with MiniCPM-o 4.5, it improves StreamingBench and OVO-Bench accuracy by 2.3 and 4.0 percentage points, respectively, and OmniPro probe-mode accuracy by 3.4 points. It also scores higher on Daily-Omni, OmniVideoBench, and LVOmniBench.

Scope of the comparison. Among the compared online models, Realtime-Venus-Omni achieves the highest scores on six of the eight benchmarks. Its lower scores on ProactiveVideoQA and WorldSense relative to MiniCPM-o 4.5 underscore the need to assess streaming comprehension, proactive response quality, and offline video understanding separately. We report offline baselines separately for reference and restrict the best- and second-best highlighting to online models. These results also do not isolate the contribution of individual training components, which requires controlled ablations.

Memory-augmented understanding. Table 3 reports the performance of the long-video memory module on LVOmniBench, LongVideoBench, and CGBench across different video-duration ranges. We report results for both Realtime-Venus-Omni and MiniCPM-o 4.5, comparing each backbone with and without memory augmentation. Overall, the results show that memory augmentation improves Realtime-Venus-Omni across all reported duration bins, with gains for MiniCPM-o 4.5 in several settings as well.

Table 3 Memory-augmented long-video understanding accuracy (%). Duration bins are in minutes.

Model	LVOmniBench			CGBench			LongVideoBench	
	[10, 30]	[30, 60]	[60, 90]	[40, 50]	[50, 60]	[60, 65]	[15, 40]	[40, 60]
MiniCPM-o 4.5	<u>38.58</u>	38.56	32.35	34.27	36.10	34.58	<u>52.89</u>	55.56
+ memory	38.80	39.89	38.24	44.71	45.37	43.93	51.90	55.56
Realtime-Venus-Omni	35.48	<u>42.91</u>	<u>41.18</u>	<u>47.84</u>	<u>46.63</u>	<u>49.53</u>	<u>52.89</u>	<u>57.14</u>
+ memory	36.36	43.10	47.06	48.71	48.31	50.47	53.29	61.90

Effect of memory. Memory augmentation consistently improves Realtime-Venus-Omni across the evaluated duration ranges. In particular, accuracy increases by 5.88 percentage points on the 60–90-minute subset of LVOmniBench and by 4.76 points on the 40–60-minute subset of LongVideoBench, with gains across all evaluated duration ranges of CGBench as well. For MiniCPM-o 4.5, memory also improves performance in several settings, including improvements of 9.27–10.44 points across CGBench and 5.89 points on the 60–90-minute LVOmniBench subset. The gains are consistent across all evaluated bins for Realtime-Venus-Omni, whereas MiniCPM-o 4.5 shows a decrease in the shorter LongVideoBench bin and no change in the longer bin. Thus, the benefit of memory augmentation depends on the backbone and evaluation setting.

7.4 Speech results

Audio understanding. Table 4 reports results for both frontend models on MMAU, MMAU-Pro, MMAR, and MMSU. Realtime-Venus-Audio achieves accuracies of 78.0%, 63.2%, 65.6%, and 66.0%, respectively.

The speech comparisons include Fun-Audio-Chat (Tongyi Fun Team et al., 2025), MiniCPM-o 2.6 (OpenBMB, 2025), Baichuan-Omni-1.5 (Li et al., 2025), Kimi-Audio (Ding et al., 2025), Qwen3-Omni, MiMo-Audio (Xiaomi LLM-Core Team, 2025), Step-Audio2-mini (Wu et al., 2025), and MiniCPM-o 4.5. Model configurations are listed in Tables 4 and 5.

Table 4 Audio understanding accuracy (%). Higher is better for all benchmarks.

Model	Size	MMAU ↑	MMAU-Pro ↑	MMAR ↑	MMSU ↑
Fun-Audio-Chat	8B	76.6	58.0	40.7	<u>67.8</u>
MiniCPM-o 2.6	7B	65.2	40.5	48.6	56.5
Baichuan-Omni-1.5	7B	65.6	42.9	40.7	50.6
Kimi-Audio	9B	68.4	56.6	60.8	59.3
Qwen3-Omni	30B-A3B	<u>77.5</u>	61.2	66.4	69.0
MiMo-Audio	7B	74.9	53.4	63.6	61.7
Step-Audio2-mini	7B	68.2	47.9	55.8	56.8
MiniCPM-o 4.5	9B	76.9	60.0	65.3	65.8
Realtime-Venus-Omni	9B	76.9	<u>62.0</u>	65.2	64.6
Realtime-Venus-Audio	9B	78.0	63.2	<u>65.6</u>	66.0

Task-dependent performance. Realtime-Venus-Audio achieves the highest scores among the compared models on MMAU and MMAU-Pro. On MMAR, it ranks second behind Qwen3-Omni. On MMSU, Qwen3-Omni and Fun-Audio-Chat score higher. These results indicate that the interaction-oriented frontend retains general audio understanding, while contextual acoustic reasoning and fine-grained spoken-language understanding remain areas for improvement.

Spoken question answering. Table 5 reports results for both frontend models. Realtime-Venus-Audio obtains an AlpacaEval score of 4.81 and accuracies of 83.8%, 75.7%, and 67.8% on Llama Questions, Speech TriviaQA, and Speech CMMLU, respectively.

Table 5 Spoken question-answering results. AlpacaEval reports a judge score; other columns report accuracy (%). Higher is better.

Model	Size	AlpacaEval ↑	Llama Q. ↑	TriviaQA ↑	CMMLU ↑
Fun-Audio-Chat	8B	<u>4.80</u>	83.3	68.1	<u>67.7</u>
MiniCPM-o 2.6	7B	4.42	78.0	51.8	51.4
Baichuan-Omni-1.5	7B	4.50	78.5	63.0	58.4
Kimi-Audio	9B	4.46	79.3	62.1	67.0
Qwen3-Omni	30B-A3B	4.74	<u>83.4</u>	75.9	47.8
MiMo-Audio	7B	4.60	79.7	52.8	56.7
Step-Audio2-mini	7B	4.17	75.0	57.7	67.6
MiniCPM-o 4.5	9B	4.81	81.0	75.5	59.2
Realtime-Venus-Omni	9B	4.75	83.3	73.6	67.5
Realtime-Venus-Audio	9B	4.81	83.8	<u>75.7</u>	67.8

Knowledge coverage. Realtime-Venus-Audio ties MiniCPM-o 4.5 for the highest AlpacaEval score and achieves the highest Llama Questions and Speech CMMLU accuracies among the compared models. It ranks second on Speech TriviaQA, behind Qwen3-Omni. These results cover instruction following and knowledge-based spoken question answering, but do not establish broader reasoning capability. The aggregate scores also do not distinguish errors in speech interpretation from errors in downstream answer generation.

7.5 Full-duplex results

Overlap handling (v1.5). Table 6 reports Full-Duplex-Bench v1.5 results for both frontend models. Realtime-Venus-Audio has a response rate of 0.75 under user interruption and continuation rates of 0.97, 0.88, and 0.86 under user backchannels, speech directed to others, and background speech, respectively. These scenario-specific results assess whether the model responds to interruptions while maintaining conversational continuity during other overlapping speech.

Table 6 Full-Duplex-Bench v1.5 results across four scenarios.

Model	User interruption		User backchannel		Talking to others		Background speech	
	C_RESPOND↑	C_RESUME↓	C_RESPOND↓	C_RESUME↑	C_RESPOND↓	C_RESUME↑	C_RESPOND↓	C_RESUME↑
Freeze-Omni [†]	0.72	0.12	0.07	0.80	0.58	0.25	0.62	0.25
Moshi*	0.50	0.26	0.02	0.06	0.20	0.19	0.21	0.07
Gemini 3.1 Live [†]	0.77	0.20	0.02	0.95	0.27	0.66	0.28	0.66
GPT-4o*	<u>0.78</u>	<u>0.10</u>	0.03	0.70	0.91	0.02	0.93	0.04
Joy-Duplex [†]	0.88	0.07	<u>0.01</u>	<u>0.96</u>	0.17	0.72	0.10	<u>0.85</u>
MiniCPM-o 4.5	0.60	0.36	0.00	0.95	0.18	0.79	0.16	0.82
Realtime-Venus-Omni	0.60	0.37	0.02	0.92	0.06	0.90	0.12	<u>0.85</u>
Realtime-Venus-Audio	0.75	0.23	0.00	0.97	<u>0.11</u>	<u>0.88</u>	<u>0.11</u>	0.86

Notes. Arrows indicate the preferred direction of each metric.

* Results are taken directly from the original Full-Duplex-Bench v1.5 paper (Lin et al., 2026b).

[†] Results for Joy-Duplex and Gemini 3.1 Live are taken from the JoyAI-Talker paper (Bai et al., 2026).

Interruption–continuation trade-off. Table 6 distinguishes overlaps requiring a new response from those requiring continued speech. Realtime-Venus-Audio achieves the highest continuation rates for user backchannels and background speech, and ranks second to Realtime-Venus-Omni for speech directed to others. However, its interruption-response rate is lower than those of Joy-Duplex, GPT-4o, and Gemini 3.1 Live. Thus, strong continuation does not necessarily imply strong interruption handling. Further evaluation should consider ambiguous overlaps and response-transition timing. We report the four scenarios separately because they require different actions; aggregation would require an explicit weighting scheme.

Tool use (FDB-v3). Table 7 reports Full-Duplex-Bench v3 results for Realtime-Venus with both frontends. Realtime-Venus-Omni achieves 86.0% tool selection F1, 53.1% argument accuracy, and 43.0% Pass@1, while Realtime-Venus-Audio achieves 82.0%, 52.2%, and 42.0%, respectively. The two frontends rank second and fourth, respectively, in tool selection F1. GPT-Realtime leads all three metrics at 87.6%, 68.0%, and 60.0%.

The tool-use baselines comprise the cascaded system, Gemini 2.5 Live (Google, 2025), Gemini 3.1 Live, Grok (xAI, 2025), Ultravox (Quigley, 2025), GPT-Realtime (OpenAI, 2025), and NemotronLabs VoiceChat-11B (NVIDIA, 2026).

Table 7 Full-Duplex-Bench v3 tool-use results (%). Higher is better for all metrics.

Model	Tool selection F1 $\uparrow$	Argument accuracy $\uparrow$	Pass@1 $\uparrow$
Cascaded	80.3%	56.2%	45.0%
Gemini 2.5 Live	78.6%	<u>59.3%</u>	49.0%
Gemini 3.1 Live	81.7%	58.8%	<u>54.0%</u>
Grok	79.7%	54.2%	43.0%
Ultravox	79.4%	51.3%	41.0%
GPT-Realtime	87.6%	68.0%	60.0%
NemotronLabs VoiceChat-11B	82.5%	42.2%	33.0%
Realtime-Venus-Omni	<u>86.0%</u>	53.1%	43.0%
Realtime-Venus-Audio	82.0%	52.2%	42.0%

Notes. Tool selection denotes F1. Pass@1 requires all expected tool calls, no extra calls, and correct arguments for every call.

Selection versus task completion. Both frontends remain below GPT-Realtime in argument accuracy and complete-episode success. Realtime-Venus-Audio achieves 52.2% argument accuracy and 42.0% Pass@1, compared with 53.1% and 43.0%, respectively, for Realtime-Venus-Omni. These metrics use different success criteria, so their difference cannot be interpreted as an intermediate-stage failure rate. The results motivate closer evaluation of argument grounding, retention of spoken constraints, and coordination across multiple calls. Identifying limiting factors requires examining individual execution trajectories, including failures after appropriate tool selection.

7.6 Delegate Benchmark

Benchmark construction. We construct an internal delegate benchmark to evaluate whether real-time interaction models can correctly decide when to delegate. Given a user request, the model must determine whether to handle it directly within the conversational frontend or issue a delegation request to Realtime-Venus-Harness. The benchmark evaluates delegation accuracy, including both necessary delegation and avoidance of unnecessary delegation.

The benchmark has two input modes with different levels of difficulty: omni input, which provides audio–visual observations to Realtime-Venus-Omni, and audio input, which provides audio-only observations to Realtime-Venus-Audio.

Task categories. The benchmark comprises three categories that probe complementary aspects of delegation behavior. *External capabilities* contains requests that require external capabilities and should therefore trigger delegation. *Routine interaction* contains requests that the frontend should handle directly without delegation. *Reasoning* includes both requests requiring delegation and requests that can be handled directly, testing whether the model can distinguish reasoning tasks that require delegation from those suitable for direct handling. Together, these categories evaluate delegation decisions across requests with different requirements for external assistance.

Evaluation protocol. We compare the model’s routing decision with the reference label for each request. Overall accuracy measures correct routing across the benchmark. The external capabilities category reports delegation recall, measuring how reliably the model identifies requests requiring delegation. The routine interaction category reports non-delegation specificity, measuring how reliably it avoids unnecessary delegation. The reasoning category reports routing accuracy within the mixed category. These metrics jointly characterize delegation decision accuracy and distinguish missed delegation from unnecessary delegation.

Results. Table 8 reports the performance of Realtime-Venus-Omni and Realtime-Venus-Audio on our delegate benchmark. Realtime-Venus-Omni achieves an overall routing accuracy of 75.93%, exceeding Realtime-Venus-Audio by 7.04 percentage points. The two variants exhibit different strengths: Realtime-Venus-Audio achieves higher delegation recall in the external capabilities category (92.22% versus 68.33%), whereas Realtime-Venus-Omni achieves higher non-delegation specificity in the routine interaction category (84.44% versus 39.44%). Both variants achieve 75.00% routing accuracy in the reasoning category.

Table 8 Delegation decision performance on our in-house Delegate Benchmark (%).

Model	Overall	External capabilities	Routine interaction	Reasoning
Realtime-Venus-Omni	75.93	<u>68.33</u>	84.44	75.00
Realtime-Venus-Audio	<u>68.89</u>	92.22	<u>39.44</u>	75.00

Notes. Overall and reasoning: routing accuracy; external capabilities: delegation recall; routine interaction: non-delegation specificity. Higher is better for all metrics.

Discussion. The benchmark identifies different delegation failure patterns in the two models. Realtime-Venus-Audio recognizes most requests requiring external capabilities but frequently delegates routine requests that should be handled locally. Realtime-Venus-Omni more reliably avoids unnecessary delegation, but misses a larger proportion of requests requiring external capabilities. These results highlight the need to improve both the recognition of delegation requirements and direct handling of requests when delegation is unnecessary. The benchmark evaluates the correctness of delegation decisions; successful execution of delegated tasks and integration of their results into the ongoing conversation require additional evaluation.

8 Conclusion and Future Work

Across the evaluated settings, Realtime-Venus demonstrates **competitive multimodal understanding** and strong conversational continuity under non-interruptive speech. Its asynchronous design provides **access to external capabilities while keeping interaction active**: background reasoning and tool execution proceed while the frontend continues receiving inputs and managing speech. Memory augmentation further supports **hour-scale video understanding**, with improvements across all evaluated duration bins. Together, these findings support coordinating immediate conversational responses with longer-running computation as a promising direction for assistants that remain engaged with users while handling tasks beyond the frontend’s own capabilities.

Future work will explore **finer-grained streaming chunks** to better capture brief events and improve the timing of conversational responses. We will extend the context window to support **longer interactions**, retaining relevant information and tracking evolving user intent. We will also investigate **more complex and diverse tasks** that require multi-step reasoning, coordinated tool use, and asynchronous execution, with particular attention to managing concurrent operations and recovering from delayed or failed tool calls.

Appendix

A Contributions

Core contributors. Ruixiang Zhao*, Hualei Wang*, Renhe Sun*, Enzhi Zhou*, Zihang Liu*, Jincenzi Wu, Xujie Song, Kexin Shi.

Contributors. Pengcheng Zhu, Jiayi Zhou, Baoyue Zhang, Changhao Zhang, Yuqian Ying, Yongxiang Xie, Zitong Wang, Jinhong Wang, Tong Niu, Jingjing Liu, Junan Lin, Haolin He, Hengshuo Chu, Yuhui Chen.

Project leaders. Jian Liu[†], Yuge Huang[†], Junliang Xing[†], Yuntao Wang[†].

Project advisor. Weiqiang Wang, Chun Yu, Yuanchun Shi.

B Training Data Example

We present one training example for each of the two data families central to this report: proactive duplex interaction and delegate workflows. Both are rendered from raw training records. In the unit-level format of Section 4.1, each `<unit>` contains one second of aligned visual (`<image>`) and audio (`<audio>`) input. The assistant responds with `<listen>` for silent perception or `<|speak|>` followed by chunked text ending with `<|chunk_eos|>`; `<|turn_eos|>` marks the end of a turn. Long runs of identical `<|listen|>` units are omitted.

B.1 Omni-Proactive Example

The assistant receives an instruction at the beginning of the stream and must fire the instant the requested event occurs, staying silent both before and after.

```
{ "role": "user", "content": "<unit>\n<image>\n<audio>\n Let me know when the commentator reacts to a wicket." }
{ "role": "assistant", "content": "<|listen|>" }
: (4 consecutive listen units omitted)
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>The commentator just reacted<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|> to the wicket<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>, shouting 'Oh<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>, he's got<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|> him!<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|><|turn_eos|><|chunk_eos|>" }
```

* Indicates equal contribution. Names marked with an asterisk are listed first; within the marked and unmarked groups, names are ordered by last name in reverse alphabetical order.

[†] Corresponding authors: Jian Liu and Yuge Huang ({rex.lj, huangyuge.hyg}@antgroup.com); Junliang Xing and Yuntao Wang ({jlxing, yuntaowang}@tsinghua.edu.cn).

: (7 trailing listen units omitted)

The example supervises the proactive response capability of Realtime-Venus-Omni: the model must remain silent while nothing relevant occurs and speak immediately when the triggering phrase appears in the audio stream.

B.2 Delegate Example

The example shows the full delegate cycle of Section 5 within a continuous video session.

```
{ "role": "user", "content": "<unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|listen|>" }

: (39 consecutive listen units omitted)
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>\n 今天北京限行尾号是啥,我这京牌车尾
号7能开吗?" }
{ "role": "assistant", "content": "<|speak|>好,<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>我查<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>一下今天的<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>限行<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>规则。<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|><delegate>查询今天北京限行尾号及京牌尾号7能否通
行。</delegate><|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|><|turn_eos|><|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|listen|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|listen|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|listen|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>\n<backend>你刚才问的北京限行查到
了: 今天限行尾号为2和7,你的京牌尾号7在五环路以内道路限行,时段是7点到20点。</backend>" }
{ "role": "assistant", "content": "<|speak|>你刚才问的<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>北京限行查<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>到了: 今天限<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>行尾号为<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>2和7,<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>你的京牌尾<|chunk_eos|>" }
{ "role": "user", "content": "</unit><unit>\n<image>\n<audio>" }
{ "role": "assistant", "content": "<|speak|>号7在五<|chunk_eos|>" }
```



```

{"role": "user", "content": "</unit><unit>\n<image>\n<audio>"}
{"role": "assistant", "content": "<|speak|>环路以内道路<|chunk_eos|>"}
{"role": "user", "content": "</unit><unit>\n<image>\n<audio>"}
{"role": "assistant", "content": "<|speak|>限行,时段<|chunk_eos|>"}
{"role": "user", "content": "</unit><unit>\n<image>\n<audio>"}
{"role": "assistant", "content": "<|speak|>是7点到<|chunk_eos|>"}
{"role": "user", "content": "</unit><unit>\n<image>\n<audio>"}
{"role": "assistant", "content": "<|speak|>20点。<|chunk_eos|>"}
{"role": "user", "content": "</unit><unit>\n<image>\n<audio>"}
{"role": "assistant", "content": "<|speak|><|turn_eos|><|chunk_eos|>"}
: (trailing listen units omitted)

```

The first response chunks acknowledge the spoken query and commit to looking it up; the `<delegate>` span dispatches the task to the Harness and the turn closes with `<|turn_eos|>`, so the frontend returns to listening while the backend executes. When the `<backend>` result re-enters the session, the model re-presents it in chunked spoken form within the same conversational timeline.